Sebastian Stohn | 8/03/26

1. Briefly describe the artifact.

The original artifact was the portfolio item for CS-320 completed in February of 2026. It was written in Java and contains the classes Contact.java and ContactService.java. Together these classes handle the creation, alteration, and deletion of Contacts and Contact fields.

TOP OF CONTROLLER CLASS

```java
/**
 * ContactController class for HTTP request routing and and API end points.
 *
 * @author Sebastian Stohn
 * @since 2026-08-03
 */
@RestController
@RequestMapping("/contacts")
public class ContactController {
    private final ContactService contactService; // Service instance

    /**
     * Initialize controller with service instance.
     *
     * @param cs Contact service instance.
     */
    public ContactController(ContactService cs) {
        this.contactService = cs;
    }

    /**
     * Request routing for addContact.
     *
     * @param contact Newly created contact.
     */
    @PostMapping
    public void addContact(@RequestBody Contact contact) {
        contactService.addContact(contact);
    }

    /**
     * Request routing for deleteContact.
     *
     * @param id Selected contact's id.
     */
    @DeleteMapping("/{id}")
    public void deleteContact(@PathVariable String id) {
        contactService.deleteContact(id);
    }
```

2. Justify the inclusion of the artifact in your ePortfolio.

I selected this artifact because it contains sound business logic that could easily be expanded from an isolated service into a Spring Boot web application with browser UI. I have made the following improvements:

- The existing Contact and ContactService classes were moved into a Maven project which was upgraded to use Spring Boot. The service layer was added by annotating ContactService with @Service. A ContactApp entry point and a ContactController were created to expose the REST endpoints. A helper method was added to ContactService to reduce repeated code and a

TOP OF INDEX.JS

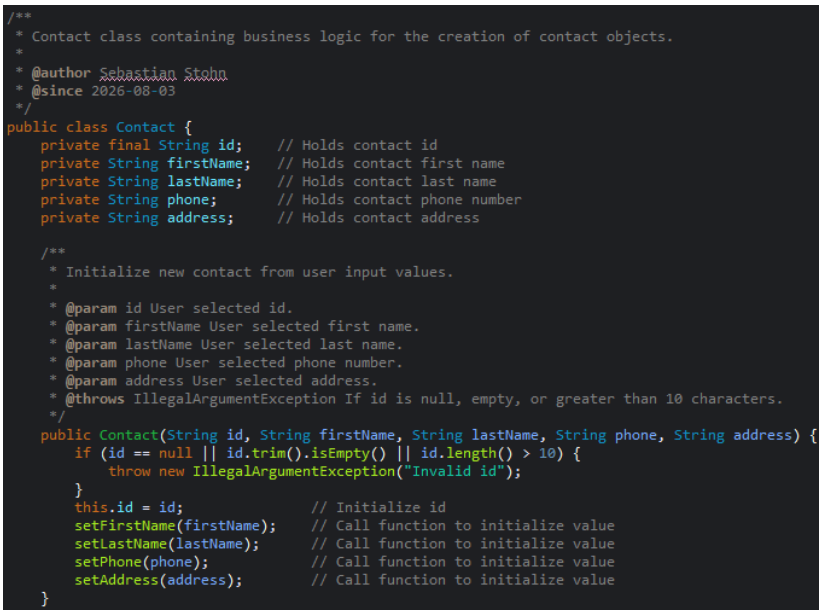
```
/**
 * @fileoverview    Handles client-side functions and UI updates for contact service.
 * @author          Sebastian Stohn
 * @date            2026-08-03
 */


/**
 * Function to create and add new contact based on contact form fields.
 */
async function addContact() {
    // Create contact from contact form
    const contact = {
        id: document.getElementById("contactId").value,
        firstName: document.getElementById("contactFirst").value,
        lastName: document.getElementById("contactLast").value,
        phone: document.getElementById("contactPhone").value,
        address: document.getElementById("contactAddress").value
    };

    // Send POST request
    const response = await fetch("/contacts", {
        method: "POST",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify(contact)
    });

    // Validate request success
    if (response.ok) {
        clearResult();
        alert("Contact sucessfully added");
    }
    else {
        clearResult();
        alert(await response.text());
    }
}
```

ContactExceptionHandler class was written to provide more meaningful error messages. Validation within the Contact class also was improved to reject empty inputs that were causing bugs.

•A basic user interface was also developed using HTML, CSS, and JavaScript with fetch() calls so users can add, find, and delete contacts through the API. Finally, update functionality was added for each mutable contact field resulting in a complete CRUD application that takes full advantage of the original service.

- Starting off the second round of enhancements, the JavaScript was then streamlined with helper functions to reduce repeated code. The HTML was also enhanced by adding placeholder text to input fields to provide users with expected value formats.
- A getAllContacts() method was added to the service layer and exposed through a new REST endpoint in the controller. The JavaScript was updated with a matching getAllContacts() function that retrieves the complete list of contacts and displays it to the user. Finally, a corresponding "Show All Contacts" button was added to the HTML.

TOP OF CONTACT CLASS



```
/**
 * Contact class containing business logic for the creation of contact objects.
 *
 * @author Sebastian Stohn
 * @since 2026-08-03
 */
public class Contact {
    private final String id;    // Holds contact id
    private String firstName;   // Holds contact first name
    private String lastName;    // Holds contact last name
    private String phone;       // Holds contact phone number
    private String address;     // Holds contact address

    /**
     * Initialize new contact from user input values.
     *
     * @param id User selected id.
     * @param firstName User selected first name.
     * @param lastName User selected last name.
     * @param phone User selected phone number.
     * @param address User selected address.
     * @throws IllegalArgumentException If id is null, empty, or greater than 10 characters.
     */
    public Contact(String id, String firstName, String lastName, String phone, String address) {
        if (id == null || id.trim().isEmpty() || id.length() > 10) {
            throw new IllegalArgumentException("Invalid id");
        }
        this.id = id;               // Initialize id
        setFirstName(firstName);    // Call function to initialize value
        setLastName(lastName);      // Call function to initialize value
        setPhone(phone);            // Call function to initialize value
        setAddress(address);        // Call function to initialize value
    }
```

These improvements have turned the raw business logic classes into a full architecture. This demonstrate an ability to build and connect multiple layers of an application while also incorporating RESTful APIs. It also showcases an understanding of HTML, CSS, and Javascript and how they interact with the backend of the application.

3. Did you meet the course outcomes you planned to meet with this enhancement?

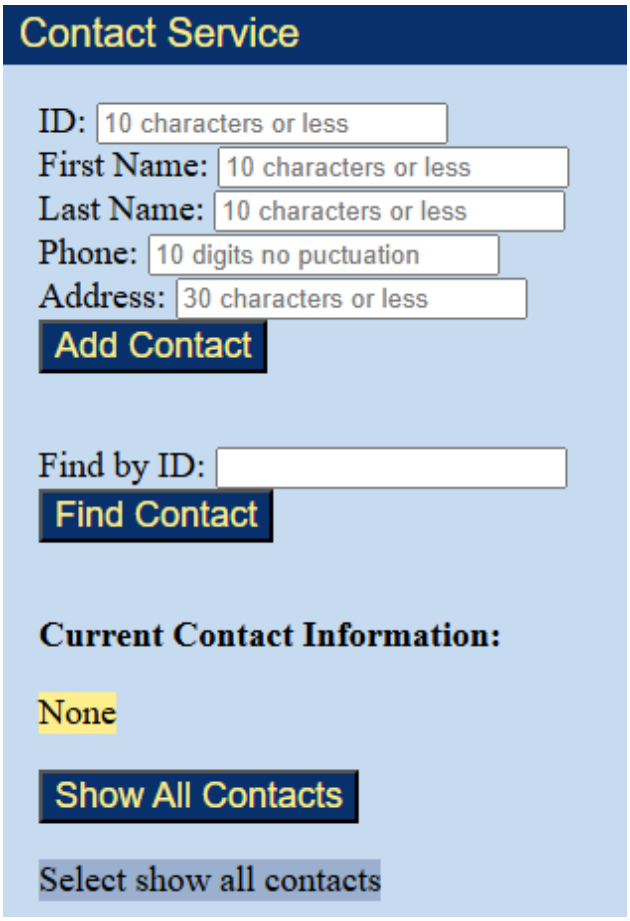
The outcome I set out to meet with this category was: “Demonstrate an ability to use well-founded and innovative techniques, skills, and tools in computing practices for the purpose of implementing computer solutions that deliver value and accomplish industry-specific goals.” I believe I’ve met this outcome because I have created a functional program using well-founded techniques while also using my own skills to solve problems and create solutions specific to this application.

As far as remaining improvements, the JavaScript could still be streamlined. However, the entire codebase has now been thoroughly commented to improve maintainability. I’ve also added headers to the top of every file to increase the project’s professionalism and aid future developers.

4. Reflect on the process of enhancing and modifying the artifact.

When updating the project to take advantage of Spring Boot I was reminded of how much work goes into just setting up the web environment for a few classes to run on. Looking further into error messages was a good experience since I learned how to give the user more feedback to make the app easier to use. I got to implement my HTML/CSS/JS skills I learned in Web Site Design while also gaining more experience integrating Javascript. I ran into several challenges during these enhancements:

LANDING PAGE



•The first challenge was that the contact class was allowing empty fields to be passed into contact objects because it didn't understand the difference between NULL and an empty string. This problem was solved by adding an additional check to the contact class.

•Another challenge was dealing with hiding and unhiding certain UI objects in HTML and JS. I had little experience with that particular skill so learning more about those functions was extremely helpful.

- The final challenge was making the UI make sense based on the most recent user input. This mostly involved "clearing the decks" of the contact that the user was currently working with

FUNCTIONALITY DEMO

when they performed a different action or make a bad input. Making it follow logical sense while still performing correctly was a good test for me.

- This challenge continued during the polishing phase as I added more API endpoints and continued to add functionality to the front-end.